

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
DO ESTADO DE MINAS GERAIS
CAMPUS DIVINÓPOLIS

ENGENHARIA DE COMPUTAÇÃO

SISTEMAS OPERACIONAIS

Documentação Final

04 de Outubro, 2026



Resumo

Este trabalho apresenta um simulador de máquina RISC (*Reduced Instruction Set Computer*) de 16 bits, implementado em C++, com execução sequencial de programas em lote. A máquina foi projetada *bare-metal*, sem escalonamento de processos, gerenciamento avançado de memória, múltiplas CPUs ou memória cache. Nesta etapa reúne oito registradores de uso geral, Unidade de Controle, Unidade Lógica e Aritmética, memória principal e um conjunto de nove instruções. O subsistema de carga lê arquivos hexadecimais, posiciona instruções e dados na RAM, reinicia o estado da CPU, acompanha a execução e gera relatórios de estado e consumo de ciclos.

Palavras-chave: RISC; arquitetura de computadores; sistemas operacionais; simulador; processamento em lote; Assembly; memória; CPU.

Sumário

1	Introdução	3
2	Arquitetura do Sistema	3
2.1	Organização da Arquitetura	3
3	Processador (CPU)	4
3.1	Organização Estrutural e Módulos	4
3.2	O Ciclo de Instrução	5
3.3	Conjunto de Instruções e Formato das Palavras	5
4	Memória e Armazenamento	6
4.1	Interfaces de Leitura, Escrita e Proteção (<i>SEGFAULT</i>)	6
4.2	Simulação do Disco Secundário e Penalidade de I/O	7
4.3	Métricas Globais de Desempenho e Monitorização	7
5	Clock e Loader	8
5.1	Fluxo de Execução em Lote (<i>Batch Lifecycle</i>)	8
5.2	Motor de Relógio (<i>ClockCore</i>) e Penalidades Temporais	10
5.3	Proteção de Memória do SO e Tolerância a Falhas	10
5.4	Estruturas de Dados e Formatação do <code>output.dat</code>	11
6	Resultados e Discussão	11
6.1	Validação Unitária e Integração dos Módulos	11
6.2	Execução em Lote e Avaliação dos Programas de Teste	12
6.3	Análise do Custo Temporal e Persistência	13
6.4	Tolerância a Falhas e Isolamento do Kernel	14
7	Conclusão	14
	Referências	15

1 Introdução

O processamento em lote organiza a execução sequencial de programas previamente submetidos, denominados *jobs*. Nesse modelo, cada programa é carregado, executado e finalizado antes do processamento do seguinte.

A máquina simulada opera em um modelo *bare-metal* simplificado, sem um sistema operacional convencional executando sobre o hardware simulado. O gerenciamento dos *jobs* é realizado diretamente pelos componentes responsáveis pelo processamento em lote.

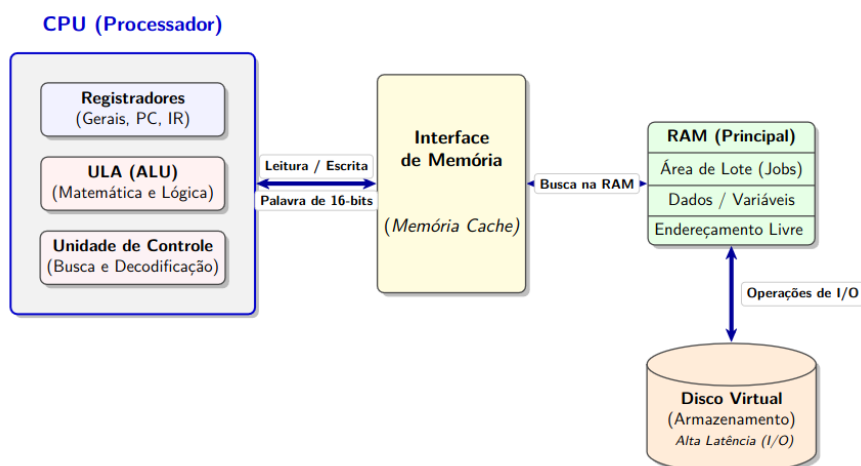
O projeto foi desenvolvido com base na especificação da prática proposta para a implementação de uma máquina RISC de 16 bits [1]. Os *jobs* são representados por arquivos de texto contendo instruções em formato hexadecimal e executados em uma máquina simulada, implementada em C++. O sistema contabiliza instruções executadas, acessos à memória, acessos ao disco e ciclos de *clock*, armazenando os resultados no arquivo `output.dat`. O código-fonte do projeto está disponível no GitHub [2].

2 Arquitetura do Sistema

2.1 Organização da Arquitetura

A arquitetura simulada é composta por uma Unidade Central de Processamento (CPU), uma memória principal, uma representação de memória secundária e mecanismos de Entrada e Saída (I/O). A Figura 1 apresenta uma representação conceitual dos principais componentes e de sua organização no simulador.

Figura 1: Arquitetura modular do simulador.



3 Processador (CPU)

O módulo da CPU segue a arquitetura **RISC** (*Reduced Instruction Set Computer*) baseada em palavras e barramentos de dados de 16 bits. O funcionamento do sistema é baseado no paradigma **Load/Store**: a CPU não processa dados diretamente na memória RAM, ocorrendo as operações matemáticas e lógicas exclusivamente entre os registradores internos. O acesso à memória externa é realizado unicamente através de instruções dedicadas de leitura (*Load*) e escrita (*Store*).

Para assegurar a conformidade com a arquitetura de 16 bits, utiliza-se o tipo `uint16_t` para barramentos, registradores e endereços. Em operações que exigem aritmética com sinal (como subtrações ou comparações), realiza-se a conversão explícita para `int16_t`, recorrendo a tipos intermediários de 32 bits (`int32_t`) para a verificação e detecção de *overflow*.

3.1 Organização Estrutural e Módulos

A arquitetura interna do subsistema de CPU encontra-se encapsulada no diretório `src/cpu/` do projeto e subdivide-se em três módulos principais:

- **Unidade de Controle (UC):** Responsável por coordenar o fluxo de execução do sistema e gerenciar o ciclo de instrução sem realizar cálculos matemáticos. A UC procura as instruções na memória, realiza a decodificação dos *opcodes* e envia sinais de comando para a ULA ou para a Interface de Memória. Adicionalmente, desvios de fluxo (**JUMP**) e instruções de parada (**HALT**) são geridos diretamente pela UC mediante a alteração do *Program Counter* ou da *flag* de estado.
- **Banco de Registradores:** Representa a estrutura de memória interna e de alta velocidade do processador. É composto por:
 - **Registradores de Uso Geral (R0 a R7):** Oito registradores de 16 bits para armazenamento temporário de operandos e variáveis. O registrador R0 comporta-se como um registrador convencional de leitura e escrita.
 - **Program Counter (PC):** Registrador responsável por armazenar o endereço da próxima instrução a ser executada na memória.
 - **Instruction Register (IR):** Registrador encarregado de armazenar a instrução atual em processo de decodificação.

A CPU disponibiliza as funções `getContext()` e `setContext()`, responsáveis pela captura e restauração do estado dos registradores. Essas funções fornecem suporte à integração com os componentes de gerenciamento do sistema e podem ser utilizadas em extensões futuras envolvendo troca de contexto.

3.2 O Ciclo de Instrução

O processamento de instruções é conduzido sequencialmente pela Unidade de Controle através de três etapas fundamentais:

1. **Busca (*Fetch*):** A CPU lê o endereço contido no registrador PC para acessar à memória RAM, copia a instrução correspondente para o registrador IR e incrementa o PC.
2. **Decodificação (*Decode*):** A instrução de 16 bits presente no IR é fracionada via mascaramento de bits (*bit masking*). A UC identifica o código de operação (*Opcode*) e os seus operandos (registradores ou valores imediatos).
3. **Execução (*Execute*):** A instrução é efetivamente processada. A UC aciona a ULA para cálculos, efetua leituras/escritas na RAM via *Load/Store*, ou atualiza o PC em caso de saltos lógicos.

3.3 Conjunto de Instruções e Formato das Palavras

Para manter a coerência em todo o simulador, os *opcodes* e constantes são definidos de forma centralizada em um arquivo. O conjunto de instruções pode ser visualizado na Tabela 1

Tabela 1: Tabela do Conjunto de Instruções (Opcodes)

Mnemónico	Opcode (Hex)	Categoria	Descrição
LOAD	0x1	Memória	Carrega um valor da RAM para um registrador interno.
ADD	0x2	ULA	Realiza a soma de dois registradores.
STORE	0x3	Memória	Salva o valor de um registrador na RAM.
SUB	0x4	ULA	Realiza a subtração entre dois registradores.
AND	0x5	Lógica	Operação lógica bit-a-bit AND.
OR	0x6	Lógica	Operação lógica bit-a-bit OR.
BEQ	0x7	Fluxo	<i>Branch if Equal</i> : Desvia a execução se dois registradores forem iguais.
JUMP	0x8	Fluxo	Salto Incondicional: Altera o PC para o endereço informado.
HALT	0xF	Sistema	Finaliza a execução do <i>Job</i> .

A palavra de 16 bits altera a sua estrutura de acordo com a categoria da instrução:

- **Instruções Aritméticas e Lógicas (ADD, SUB, AND, OR):**
[Opcode: 4 bits] [Destino: 4 bits] [Fonte 1: 4 bits] [Fonte 2: 4 bits]
- **Instruções de Transferência e Condicionais (LOAD, STORE, BEQ):**
[Opcode: 4 bits] [Destino: 4 bits] [Endereço de Memória: 8 bits]
- **Instrução de Salto Incondicional (JUMP):**
[Opcode: 4 bits] [Endereço de Memória: 12 bits]

4 Memória e Armazenamento

O subsistema de memória principal é constituído por um vetor contíguo de posições de 16 bits, totalizando 65 536 endereços físicos disponíveis no sistema (TAM_RAM). A gestão da memória estabelece um limite de proteção rígido para garantir a integridade do sistema operativo:

- **Área Reservada ao Sistema Operativo:** Os primeiros 512 endereços de memória (0 a 511), definidos pela constante `OS_RESERVED_MEM`, destinam-se exclusivamente às rotinas do sistema operativo.
- **Área do Usuário:** As posições de memória a partir do endereço 512 até 65 535 encontram-se disponíveis para o carregamento e execução dos programas do usuário (*Jobs*).

4.1 Interfaces de Leitura, Escrita e Proteção (*SEGFAULT*)

O acesso à memória RAM é mediado pelas funções `LerMemoria` e `EscreverMemoria`, garantindo a abstração do barramento e o controlo de acessos indevidos:

- **Verificação de Limites e Exceções:** Qualquer tentativa de leitura ou escrita efetuada em endereços inferiores a `OS_RESERVED_MEM` (< 512) interrompe imediatamente a operação e lança uma exceção `std::runtime_error` com a mensagem "SEGFAULT: Tentativa de leitura/escrita em area restrita do SO!".
- **Consumo de Ciclos de Relógio:** Cada operação válida de leitura ou escrita na memória RAM contabiliza uma penalidade de 1 ciclo no relógio global do sistema (`clockGlobal`).
- **Controlo Estatístico:** A cada acesso efetuado com sucesso à RAM, o contador estatístico `acessosMemoria` é incrementado unitariamente.

Tabela 2: Assinaturas e Comportamento da Memória Principal

Função	Parâmetros	Custo (Clocks)	Exceção Lançada
LerMemoria	uint16_t endereco	1 ciclo	std::runtime_error (SEGFAULT)
EscreverMemoria	uint16_t endereco, uint16_t dado	1 ciclo	std::runtime_error (SEGFAULT)

4.2 Simulação do Disco Secundário e Penalidade de I/O

Para simular o comportamento de um dispositivo de armazenamento secundário, é disponibilizada a interface `LerDisco`. O disco é representado por uma estrutura vetorial (`std::vector<uint16_t>`) na qual a procura de dados ocorre de forma sequencial:

- **Busca Sequencial de Dados:** A função percorre os setores do disco à procura do valor indicado por `dadoBuscado`, retornando o índice do setor onde o dado foi localizado, ou `-1` caso o mesmo não exista no dispositivo.
- **Penalidade Temporal de I/O:** Para modelar a latência física do dispositivo de armazenamento, cada setor percorrido durante a busca adiciona uma penalidade estática de 50 ciclos ao relógio global do sistema (`clockGlobal += 50`).
- **Registro de Operações de Disco:** Cada chamada à função `LerDisco` incrementa o contador global `acessosDisco`.

4.3 Métricas Globais de Desempenho e Monitorização

O subsistema de memória partilha variáveis globais com modificador de ligação externa (`extern`) para permitir a recolha e análise estatística da execução por parte do simulador:

- `clockGlobal`: Acumulador do total de ciclos de relógio consumidos pelas instruções e operações de I/O.
- `instExecutadas`: Contador do número total de instruções efetivamente processadas pela CPU.
- `acessosMemoria`: Registador do número total de operações de leitura e escrita realizadas na memória RAM.
- `acessosDisco`: Registador do número total de pesquisas/consultas acionadas no armazenamento secundário.

5 Clock e Loader

O subsistema de *Clock* e *Loader* é responsável pelo ciclo de vida completo dos programas do usuário (*Jobs*), gerindo o processamento em lote (*batch*), a alocação de memória principal, o acompanhamento temporal de ciclos de relógio e a persistência de telemetria. A arquitetura modular divide-se em sete componentes hierárquicos isolados no diretório `loader/`:

- **BatchManager** (`batch_manager.hpp/.cpp`): Orquestrador global encarregado de gerenciar a fila de execução em lote, inicializar contextos e capturar exceções durante o ciclo de vida dos *Jobs*.
- **JobParser** (`parser.hpp/.cpp`): Módulo responsável pela leitura analítica dos arquivos de texto (`.txt`), ignorando comentários e linhas em branco, e pela extração e conversão de instruções e dados para valores de 16 bits (`uint16_t`).
- **ClockCore** (`clock_core.hpp/.cpp`): Motor do relógio do sistema responsável por acumular os ciclos de relógio e aplicar as penalidades temporais associadas a acessos à memória RAM e operações de disco.
- **RAMLoader** (`ram_loader.hpp/.cpp`): Componente encarregado do mapeamento físico e da cópia das instruções e variáveis para a memória RAM a partir do endereço de base predefinido.
- **MetricsTracker** (`metrics.hpp/.cpp`): Sistema de telemetria responsável por registrar o volume de instruções executadas, o número de acessos à RAM/disco e os estados finais da CPU para cada *Job*.
- **Reporter** (`reporter.hpp/.cpp`): Módulo de formatação e exportação dos resultados, encarregado de apresentar dados no terminal e gravar os registros no arquivo persistente `output.dat`.
- **clock_types.hpp**: Definição centralizada de tipos de dados e estruturas de telemetria (`JobData` e `JobMetrics`).

5.1 Fluxo de Execução em Lote (*Batch Lifecycle*)

A execução dos programas em lote é coordenada sequencialmente pelo **BatchManager**, que atua como elemento central de integração entre o processamento dos *Jobs*, o carregamento da memória e a execução da CPU. Para cada programa submetido, o sistema percorre cinco etapas principais, desde a leitura do arquivo de entrada até o armazenamento das estatísticas finais de execução.

1. **Análise do arquivo (*Parsing*):** O *JobParser* realiza a leitura do arquivo associado ao *Job*, identificando e separando as seções de código (*.text*) e de dados (*.data*). As informações obtidas são organizadas na estrutura *JobData*.
2. **Reinicialização do contexto (*Reset*):** Antes da execução de cada novo *Job*, o estado utilizado pela execução anterior é completamente reinicializado. São zerados os contadores de ciclos, acessos à memória, acessos ao disco e instruções executadas. A CPU também é restaurada ao estado inicial, com o *Program Counter* definido como 512 e os registradores R0–R7 zerados.
3. **Carregamento na RAM (*RAM Loader*):** Após a preparação dos dados, o código do *Job* é transferido para a memória principal a partir do endereço base 512. Os dados da seção *.data* são posicionados conforme os endereços relativos especificados pelo programa.

A sequência dessas etapas e a interação entre os principais componentes responsáveis pelo processamento podem ser observadas na Figura 2.

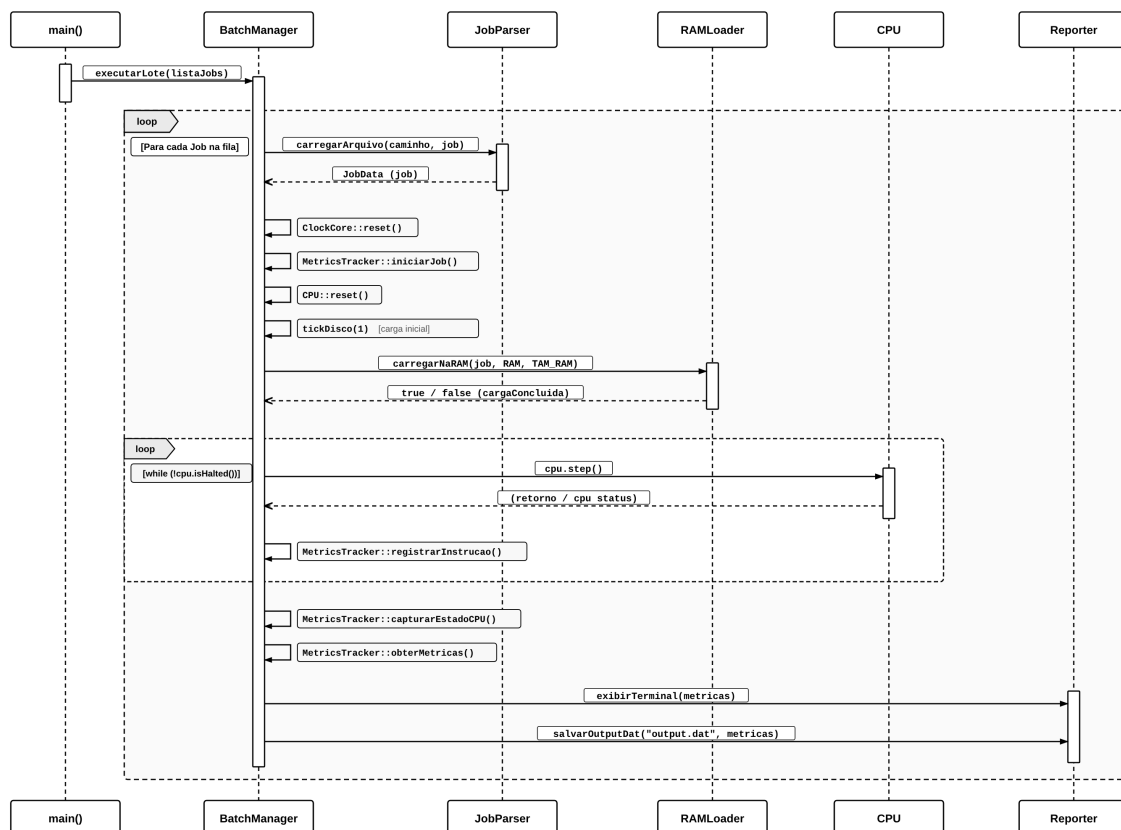


Figura 2: Fluxo de execução de um *Job* no processamento em lote.

A partir do carregamento do programa, o processamento é assumido pela CPU. O `BatchManager` inicia o ciclo de execução por meio de chamadas sucessivas a `cpu.step()`, que realizam a busca, decodificação e execução das instruções.

4. **Execução pela CPU (*Execution Loop*):** O processo continua até que seja encontrada a instrução `HALT`, correspondente ao *opcode* `0xF`, ou até que seja atingido o limite de segurança de 1 000 000 de instruções.
5. **Persistência e geração do relatório (*Reporting*):** Ao término da execução, o estado final da CPU e as métricas acumuladas são registrados em um *snapshot*. Essas informações são exibidas no terminal e armazenadas no arquivo `output.dat`.

5.2 Motor de Relógio (*ClockCore*) e Penalidades Temporais

O `ClockCore` contabiliza a latência temporal das operações do simulador, aplicando penalidades específicas com base no tipo de acesso realizado:

Tabela 3: Modelagem de Latência e Custos de Relógio

Operação	Custo (Ciclos)	Método do <code>ClockCore</code>
Leitura ou escrita na memória RAM	+1 ciclo	<code>ClockCore::tickRAM()</code>
Busca de instrução (<i>Fetch</i>)	+1 ciclo	<code>ClockCore::tickBusca()</code>
Acesso ao disco secundário (por setor)	+50 ciclos/setor	<code>ClockCore::tickDisco(n)</code>
Cálculo interno na ULA (<code>ADD</code> , <code>SUB</code> , etc.)	0 ciclos adicionais	Executado no ciclo da instrução

O cálculo total de ciclos consumidos por um *Job* é dado por:

$$\text{Clock Total} = (\text{Acessos RAM} \times 1) + (\text{Setores Disco} \times 50) + \text{Ciclos Extras} \quad (1)$$

5.3 Proteção de Memória do SO e Tolerância a Falhas

A faixa de endereços entre `0x0000` e `0x01FF` (0 a 511) é reservada ao Sistema Operativo (`OS_RESERVED_MEM`). Tentativas ilícitas de acesso por parte de um *Job* acionam exceções de violação de memória (*SEGFAULT*).

O `BatchManager` captura essas exceções, assinala a *flag* `erroKernel = true`, interrompe a execução do *Job* defeituoso e retoma automaticamente a execução do *Job* seguinte da fila. Esta capacidade de recuperação assegura a resiliência do processamento em lote em ambientes com falhas individuais.

5.4 Estruturas de Dados e Formatação do `output.dat`

O sistema utiliza duas estruturas principais para a passagem de informação e recolha de estatísticas:

- **JobData:** Armazena o identificador (`id`), o caminho do arquivo (`nomeArquivo`), o vetor de instruções decodificadas (`instrucoes`) e a lista de pares de variáveis (`variaveis`).
- **JobMetrics:** Mantém os contadores de desempenho (`instrucoesExecutadas`, `acessosRAM`, `acessosDisco`, `clockTotal`), os estados finais do processador (`pcFinal`, `irFinal`, `registradores[8]`) e o indicador de erro (`erroKernel`).

O arquivo `output.dat` é atualizado em modo de anexo, na pasta inicial do repositório, estruturando cada registro com a seguinte especificação visual:

```
--- ESTADO DA CPU ---
PC: 0x0205 IR: 0xF000
Registradores Gerais:
R0: 0x0005   R1: 0x0007   R2: 0x000C   R3: 0x0000
R4: 0x0000   R5: 0x0000   R6: 0x0000   R7: 0x0000
--- ESTATISTICAS DO JOB ---
Instrucoes Executadas: 5
Acessos a Memoria (RAM): 8
Acessos ao Disco (I/O): 1
CLOCK TOTAL DESTE JOB: 58 ciclos
=====
```

6 Resultados e Discussão

6.1 Validação Unitária e Integração dos Módulos

A verificação do simulador foi realizada por meio de testes unitários dos componentes de hardware e testes integrados entre memória e CPU.

No módulo de memória (`teste_memoria.cpp`), foram validados o isolamento da área reservada ao Sistema Operacional, as operações de leitura e escrita na memória de usuário (512 a 65,535), o incremento do relógio a cada acesso à RAM e a busca sequencial em disco, com penalidade de 50 ciclos por setor percorrido. Durante esses testes, identificou-se um *overflow* em `TAM_RAM`, inicialmente declarado como `uint16_t` com valor 65,536. A alteração para `uint32_t` corrigiu o problema e permitiu a alocação das 65,536 palavras previstas pela arquitetura.

Na integração entre CPU e memória (`teste_cpu_memoria.cpp`), foram verificadas as etapas de busca, decodificação e execução, incluindo o correto funcionamento das instruções `LOAD` e `STORE` com o deslocamento da área de usuário (`OS_RESERVED_MEM = 512`). Também foi validado o tratamento de instruções inválidas, que geram uma exceção de hardware e permitem a interrupção controlada do programa pelo Sistema Operacional.

Os componentes internos da CPU foram validados por meio da suíte localizada em `src/cpu/test/`, executada via `make test`:

- **Unidade Lógica e Aritmética (`test_ula.cpp`):** Validação das operações aritméticas (`ADD`, `SUB`), lógicas (`AND`, `OR`) e das *flags* de zero e estouro (*overflow*).
- **Unidade de Controle (`test_control_unit.cpp`):** Verificação do mascaramento de bits para extração dos campos de *opcode*, registradores e endereços imediatos, além do controle de fluxo nas instruções `BEQ` e `JUMP`.
- **Banco de Registradores (`test_registers.cpp`):** Comprovação de leitura e escrita nos registradores `R0` a `R7`, atualização do `PC` e `IR`, e integridade das funções de captura e restauração de contexto (`getContext/setContext`).
- **Integração da CPU (`test_cpu_integration.cpp`):** Execução conjunta da Unidade de Controle, ULA e Banco de Registradores executando um programa de teste na memória simulada.

6.2 Execução em Lote e Avaliação dos Programas de Teste

A validação de ponta a ponta da máquina foi realizada com a execução em lote coordenada pelo `BatchManager`, processando os três programas oficiais (*jobs*) fornecidos na especificação. A Tabela 4 sintetiza os resultados quantitativos obtidos ao final de cada execução. A análise funcional de cada programa confirma o cumprimento da especificação:

- **Job 1 (`job1_soma.txt`):** Executa a soma $C = A + B$, com $A = 5$ e $B = 7$. Foram executadas 5 instruções (`LOAD`, `LOAD`, `ADD`, `STORE`, `HALT`). O registrador `R2` armazenou o resultado `0x000C` (12 em decimal), que em seguida foi persistido na RAM. O ciclo de execução encerrou no endereço `PC = 0x0205`.
- **Job 2 (`job2_vetor.txt`):** Avalia operações aritméticas e lógicas com operandos $X = 20$ (`0x0014`) e $Y = 8$ (`0x0008`). Após 6 instruções, o registrador `R2` conteve o resultado da subtração $20 - 8 = 12$ (`0x000C`) e o registrador `R3` conteve o resultado da operação lógica $20 \& 8 = 0$ (`0x0000`), finalizando em `PC = 0x0206`.
- **Job 3 (`job3_loop.txt`):** Implementa uma estrutura de repetição com contagem regressiva de 3 a 0. O desvio condicional (`BEQ`) manteve a execução em laço por 3 iterações completas. Foram processadas 17 instruções no total, e o registrador contador `R3` terminou em `0x0000`, com o `PC` estacionando em `0x0209` após o `HALT`.

Tabela 4: Resultados Consolidados da Execução em Lote

Métrica de Avaliação	Job 1 (Soma)	Job 2 (Operações)	Job 3 (Laço)
Arquivo de Entrada	job1_soma.txt	job2_vetor.txt	job3_loop.txt
Instruções Executadas	5	6	17
Acessos à Memória RAM	8	9	23
Acessos ao Disco (I/O)	1	1	1
Tempo Total (Ciclos de Clock)	58 ciclos	59 ciclos	73 ciclos
<i>Program Counter</i> Final (PC)	0x0205	0x0206	0x0209
Instrução Final (IR)	0xF000 (HALT)	0xF000 (HALT)	0xF000 (HALT)
Estado dos Registradores Relevantes	R2 = 0x000C (12 decimal)	R2 = 0x000C R3 = 0x0000	R3 = 0x0000 (Contador zerado)
Status de Término	Sucesso (HALT)	Sucesso (HALT)	Sucesso (HALT)

6.3 Análise do Custo Temporal e Persistência

O tempo de execução de cada tarefa seguiu com precisão o modelo determinístico de custos definido para o simulador:

$$\text{Clock Total} = (\text{Acessos RAM} \times 1) + (\text{Setores Percorridos no Disco} \times 50) \quad (2)$$

Para cada programa submetido, o carregamento a partir do disco virtual gerou 1 acesso a disco, adicionando uma taxa inicial de 50 ciclos de relógio. Os acessos à memória RAM dividem-se estritamente entre a etapa de busca de instruções (*fetch*) e as operações de transferência de dados (LOAD e STORE):

- No **Job 1**, as 5 instruções exigiram 5 buscas na RAM, somadas a 2 leituras de variáveis e 1 escrita do resultado ($5 + 2 + 1 = 8$ acessos). O custo total resultou em $50 + 8 = 58$ ciclos.
- No **Job 2**, 6 buscas somadas a 2 leituras e 1 escrita totalizaram 9 acessos à RAM, perfazendo $50 + 9 = 59$ ciclos.
- No **Job 3**, as 17 instruções executadas ao longo das iterações exigiram 17 buscas, além de 6 acessos a dados no corpo do laço, totalizando 23 acessos à RAM e resultando em $50 + 23 = 73$ ciclos.

Ao término do lote, o módulo **Reporter** persistiu os dados de telemetria no arquivo `output.dat` em modo de acréscimo (*append*), registrando o estado final dos registradores em formato hexadecimal de 4 dígitos com zeros à esquerda e as estatísticas de desempenho de cada execução.

6.4 Tolerância a Falhas e Isolamento do Kernel

Para atestar o comportamento da máquina em situações de falha, submeteu-se o simulador a acessos em endereços protegidos (< 512). O hardware simulado interceptou a operação e gerou a interrupção por meio de exceção.

O `BatchManager` capturou o evento, marcou a tarefa com o indicador de erro e abortou sua continuidade. Constatou-se que a falha de um programa individual não desestabilizou o ambiente de execução: o contexto da CPU e os contadores de relógio foram reinicializados, e a fila de processamento prosseguiu normalmente para as tarefas seguintes, confirmando a resiliência do processamento em lote.

7 Conclusão

O desenvolvimento do simulador RISC de 16 bits permitiu implementar e integrar os principais componentes especificados para a primeira fase do projeto, incluindo CPU, memória, processamento em lote, controle de *clock* e geração de relatórios.

Os testes realizados demonstraram o funcionamento das instruções, o isolamento da área de memória reservada ao sistema, o tratamento de falhas e a consistência dos custos temporais definidos para o simulador. A execução dos três *jobs* de teste também confirmou a integração entre os módulos e a geração correta das métricas.

A arquitetura desenvolvida estabelece uma base para futuras extensões, como *pipeline*, memória *cache*, multiprogramação, escalonamento de processos e memória virtual.

Referências

- [1] SILVA, Michel Pires da. *Especificação do Simulador: Arquitetura Computacional e Sistemas Operacionais. Fase 1: Processamento em Lote e Arquitetura RISC (16-bits)*. Enunciado da prática, 7 p., 2026.
- [2] MSJUBR. *OperationalSystem*. GitHub, 2026. Disponível em: <https://github.com/msj jubr/OperationalSystem>. Acesso em: 2 out. 2026.